

CARGOFIT V1.0.4



คู่มือ Algorithm การจัดวางสินค้า ในตู้คอนเทนเนอร์

Container Packing Engine — Technical Reference

VERSION

1.0.4

ประเภท

Technical Reference

วันที่

พฤษภาคม 2569

สารบัญ

1	ภาพรวมระบบ	3
2	นิยามศัพท์สำคัญ	4
3	ข้อมูล Input ที่ระบบรับ	5
4	ลำดับขั้นตอน Algorithm	6
—	ชั้น 0: คำนวณจำนวนชั้นที่เหมาะสม (ComputeEffectiveLayers)	7
—	ชั้น 1: การจัดวางหลัก (Primary Packing)	8
—	ชั้น 2: การปรับสมดุล (Balancing)	9
—	ชั้น 3: ปรับสมดุลชั้น (Layer Balancing)	10
—	ชั้น 4: ตัดกองส่วนเกิน (Partial Removal)	10
—	ชั้น 5: เรียงกองตามความสูง	11
—	ชั้น 6: โชนหน้าตู้ (Condo Placement)	11
—	ชั้น 7: กระจายกล่องที่เหลือ (Scatter)	12
5	รูปแบบการวางชั้น (Layer Pattern)	13
6	ตัวอย่างจริง	15
7	การรับประกันและข้อจำกัด	16

CargoFit คำนวณการจัดวางกล่องสินค้าลงในตู้คอนเทนเนอร์โดยอัตโนมัติ โดยพยายามให้ได้จำนวนกล่องมากที่สุดภายใต้ข้อจำกัดด้านขนาดและความสูงที่รับน้ำหนักได้ ผลลัพธ์แสดงเป็นภาพ 3 มิติเพื่อให้ทีมขนส่งวางแผนล่วงหน้า

ข้อมูลที่ได้รับ

- ประเภทตู้คอนเทนเนอร์
- รายการสินค้า + ขนาด + Pattern
- จำนวนกล่องที่ต้องการบรรจุ

ผลลัพธ์ที่ส่งออก

- ตำแหน่ง (X, Y, Z) ของกล่องทุกกล่อง
- สถิติจำนวนที่บรรจุได้จริง
- ภาพ Isometric 3 มิติ

แกนพิกัดของตู้

มุมมองจากด้านบน - ทิศทางแกน X/Y ภายในตู้

Primary Zone

กองสินค้าหลัก — เรียงจากผนังหลัง (Y=0)

Y : 0 → L-50 cm

Condo

กล่องส่วนเกิน

~50 cm

ประตู



ระบบพิกัด: แกน **X** = ความกว้าง (ซ้าย→ขวา), แกน **Y** = ความยาว (ผนังหลัง→ประตู), แกน **Z** = ความสูง (พื้น→เพดาน)

คำศัพท์	ความหมาย
Stack (กอง)	กลุ่มกล่องซ้อนกันในแนวตั้ง ณ ตำแหน่ง Y เดียวกัน แต่ละกองมีหมายเลข <code>StackIndex</code> เริ่มจาก 0
Layer (ชั้น)	ระดับความสูงหนึ่งๆ ของกอง — ชั้น 0 = พื้น, ชั้น 1 = เหนือขึ้นไป มี <code>LayerIndex</code>
Pattern A / B	รูปแบบการจัดเรียงกล่องต่อชั้น — ชั้นเลขคู่ใช้ A, ชั้นเลขคี่ใช้ B เพื่อสลับทิศทางเพิ่มความเสถียร
Section	กลุ่มย่อยภายใน Pattern หนึ่งชั้น วางต่อกันในแนวกว้าง (X)
SubRow	แถวย่อยภายใน Section ที่มีการหมุนกล่อง (Rotated) ต่างกัน
Primary Zone	พื้นที่หลักจาก Y=0 ถึง L-50 cm สำหรับจัดกองสินค้าหลัก
Condo Zone	พื้นที่หน้าตู้ ~50 cm สำหรับวางกล่องส่วนเกินจาก Primary เป็นคอลัมน์ตรงๆ
Scatter	การนำกล่องที่เหลือหลัง Condo วางเสริมทับยอดกองเดิม (เฉพาะสินค้าชนิดเดียวกัน)
MaxLayers	ชั้นสูงสุดต่อกองที่อนุญาต กำหนดตามน้ำหนักสินค้าและความปลอดภัย
EffectiveLayers	ชั้นที่ระบบคำนวณให้ใช้จริง เพื่อกระจายหลายสินค้าให้ใช้ความยาวตู้สม่ำเสมอ ($\leq \text{MaxLayers}$)
CBM	ปริมาตรกล่องในหน่วยลูกบาศก์เมตร = $W \times L \times H \div 1,000,000$
Tier Replication	การทำซ้ำชุด Pattern ในแนวกว้าง X เมื่อตู้กว้างพอจะใส่ได้มากกว่า 1 ชุด

ตู้คอนเทนเนอร์ (ContainerSpec)

ฟิลด์	ความหมาย	ตัวอย่าง (ตู้สั้น)
Name	ชื่อประเภทตู้	ตู้สั้น
NominalW/L/H	ขนาดตามสเปก (cm)	244 × 600 × 259
Gap	ระยะห่างจากผนัง (cm)	5
InteriorW/L/H	พื้นที่ใช้งานจริง = Nominal – Gap	239 × 595 × 254



ตู้มาตรฐาน: ตู้สั้น 20 ft (244×600×259), ตู้ยาว 40 ft (244×1209×260), ตู้ไฮคิวบ์ 40 ft HC (244×1203×290)

สินค้า (ProductSpec)

ฟิลด์	ความหมาย	ตัวอย่าง
W, L, H	ขนาดกล่อง กว้าง × ยาว × สูง (cm)	26.2 × 34.8 × 26.7
PatternA	รูปแบบการวางชั้นเลขคู่ (0, 2, 4, ...)	[Section, Section]
PatternB	รูปแบบการวางชั้นเลขคี่ (1, 3, 5, ...)	[Section, Section]
MaxLayers	ชั้นสูงสุดต่อกอง (0 = เต็มความสูงตู้)	8
CondoCount	จำนวนกล่องต่อแถวในโซน Condo (0 = auto)	9

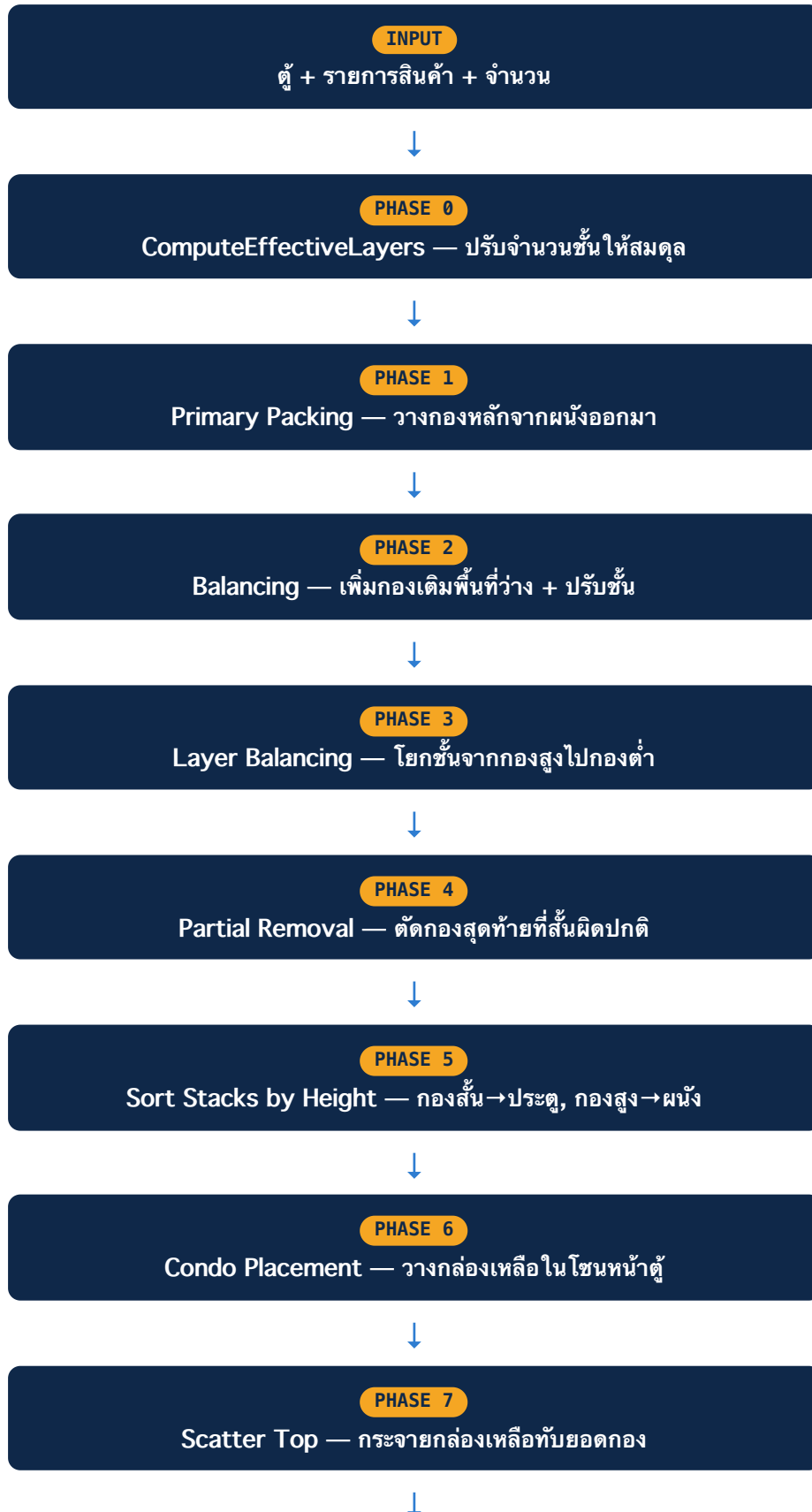


สินค้าที่ไม่มี PatternA จะไม่ถูกวางในทุกชั้นตอน — ต้องกำหนด Pattern ก่อนใช้งาน

04 ลำดับขั้นตอน Algorithm

6-Phase Packing Algorithm

Algorithm ทำงานตามลำดับ 8 ขั้นตอน (ขั้น 0-7) โดยแต่ละขั้นรับ Output จากขั้นก่อนหน้า:



เมื่อมีสินค้าหลายชนิด ถ้าปล่อยให้แต่ละชั้นวางสูงเต็ม MaxLayers พื้นที่ความยาวตู้อาจถูกใช้ไม่หมด หรืออาจสั้น ชั้นนี้หา EffectiveLayers ให้รวมพื้นที่ที่ใช้ทั้งหมดอยู่ภายใน $\text{targetY} = \text{InteriorL} - 50 \text{ cm}$

สูตรคำนวณเบื้องต้น

สำหรับสินค้าแต่ละชนิด i :

```
bpl[i]          = กว้างต่อชั้น (จาก Pattern A)
stackDepth[i]  = ความลึกของกอง 1 กอง (cm)
stacks[i]       = [ qty / (bpl × MaxLayers) ]
```

```
requiredY =  $\sum$  stacks[i] × stackDepth[i]
targetY   = InteriorL - 50 cm
```

ถ้า $\text{requiredY} \leq \text{targetY}$ → ใช้ MaxLayers ปกติ (ไม่ต้องปรับ)
ถ้า $\text{requiredY} > \text{targetY}$ → ลด EffectiveLayers ด้วย Snap Point Search

Snap Point Search

ระบบสร้างชุด snap points คือความสูงที่เป็นจำนวนเต็มเท่าของความสูงกล่อง แล้วทดลองจากน้อยไปมาก:

สินค้า A สูง 26.7 cm → {26.7, 53.4, 80.1, 106.8, ...}

สินค้า B สูง 15.7 cm → {15.7, 31.4, 47.1, 62.8, ...}

รวมทั้งหมด เรียงน้อย-มาก: {15.7, 26.7, 31.4, 47.1, 53.4, ...}

ทดลองทุก snap T_h :

```
effectiveLayers[i] = [  $T_h / H[i]$  ]
requiredY =  $\sum$  [ totalLayers[i] / effectiveLayers[i] ] × stackDepth[i]
→ เลือก  $T_h$  แรกที่  $\text{requiredY} \leq \text{targetY}$ 
```



ทำไมต้อง Snap Points? การจำกัดด้วยจำนวนเต็มชั้นทำให้กองสินค้าต่างชนิดมีความสูงใกล้เคียงกัน หากใช้การ scale แบบทั่วไปจะได้ชั้นครึ่งซึ่งในความเป็นจริงเป็นไปได้

วางกองสินค้าแต่ละชนิดเรียงจาก ด้านในสุด ($Y=0$) ออกมาหาประตู สินค้าชนิดแรกในรายการวางก่อนตามด้วยชนิดถัดไป

ขั้นตอนการวางกอง

- 1 คำนวณ `stackDepth` = ความลึกกอง 1 กอง (ผลรวมความยาวกล่องในแนว Y ตาม Pattern)
- 2 วางกองที่ `currentY` — ข้อนชั้นตาม Pattern จนครบ `EffectiveLayers` หรือเต็มความสูงตู้
- 3 เลื่อน `currentY += stackDepth` แล้ววางกองถัดไป
- 4 วนซ้ำจนได้กล่องครบหรือ `currentY ≥ InteriorL`

กฎการสลับ Pattern A/B



กองเลขคู่ (SI 0, 2, 4, ...) — ชั้น 0=A, 1=B, 2=A, ...

กองเลขคี่ (SI 1, 3, 5, ...) — ชั้น 0=B, 1=A, 2=B, ...

การสลับทิศทางทุกชั้นและทุกกองทำให้โครงสร้างล็อกกันมั่นคงขึ้นระหว่างขนส่ง

Tier Replication

ถ้าความกว้างตู้เกินกว่า 1 ชุด Pattern ระบบทำซ้ำ Pattern ในแนว X อัตโนมัติ:

```
tierWidth = ผลรวมความกว้างของทุก Section ใน Pattern
numTiers = [ InteriorW / tierWidth ]
```

ตัวอย่าง Aloe 365:

Section1 = $6 \times 21.9 = 131.4$ cm (ไม้หมุน)

Section2 = $3 \times 33.4 = 100.2$ cm (หมุน)

tierWidth = 231.6 cm, InteriorW = 239 cm → numTiers = 1

PHASE 2

การปรับสมดุล

Balancing (A + B + C)

หลัง Primary อาจยังมีพื้นที่ว่างและกล่องที่ยังไม่ได้วาง ชั้นนี้แบ่งเป็น 3 ส่วน

ส่วน A — เพิ่มกองเดิมพื้นที่ว่าง

ขณะที่ `currentY < targetY` ระบบวนเพิ่มกองใหม่ให้สินค้าที่มีกล่องเหลือ เรียงลำดับจาก CBM มากที่สุดก่อน (สินค้าใหญ่กว่ามีสิทธิ์เติมพื้นที่ก่อน)

ขณะที่ `currentY < targetY` และมีสินค้าเหลือ:
 สำหรับสินค้าเรียงตาม CBM (มาก→น้อย):
 ถ้ามีกล่องเหลือและตำแหน่งไม่ถูก clip:
 → เพิ่มกองใหม่ที่ `currentY`
 → `currentY = max(currentY, r.EndY)`

ส่วน B — ปรับจำนวนชั้นให้ตรง EffectiveLayers

กองสูงเกิน EffectiveLayers

ลบชั้นบนสุดออกจนถึงขีดจำกัด

กองต่ำกว่า EffectiveLayers

เติมชั้นขึ้นไป ถ้ากล่องเพียงพอต่อ 1 ชั้นเต็ม

ส่วน C — อัปเดตสถิติ

นับจำนวนกล่องที่วางได้จริงในพื้นที่ Primary และบันทึกไว้เพื่อขั้นตอนถัดไป

PHASE 3

ปรับสมดุลชั้น

Layer Balancing

หากกองของสินค้าชนิดเดียวกันยังมีความสูงต่างกันเกิน 1 ชั้น จะโยกชั้นบนสุดจากกองสูงที่สุดไปเพิ่มให้กองต่ำที่สุด ทำซ้ำจนความต่าง ≤ 1 ชั้น

ก่อน:



[A] [B]

A: 4 ชั้น, B: 2 ชั้น → ต่าง 2 (X)

หลัง:



[A] [B]

A: 3, B: 3 → ต่าง 0 (✓)

PHASE 4

ตัดกองส่วนเกิน

Partial Removal

ถ้าโซน Condo แน่น (พื้นที่ < 50 cm) ตรวจกองสุดท้ายของแต่ละสินค้า — ถ้ากองนั้นสั้นกว่ากองก่อนหน้าเกิน 1 ชั้น จะถูกลบออก เพื่อให้กล่องไปอยู่ใน Condo/Scatter แทนซึ่งใช้พื้นที่คุ้มกว่า

PHASE 5

เรียงกองตามความสูง

Sort Stacks by Height

จัดให้กองที่ สั้นกว่า อยู่ใกล้ประตู (Y มากกว่า) และกองที่ สูงกว่า อยู่ด้านใน (Y น้อยกว่า) ช่วยให้พนักงานมองเห็นสินค้าชั้นล่างและเข้าถึงง่ายกว่า นอกจากนี้ยังทำให้โซน Condo มีความสูงที่ถูกต้องในชั้นถัดไป

PHASE 6

การจัดวางโซนหน้าตู้

Condo Placement

วางกล่องที่ยังเหลือหลัง Primary ลงในพื้นที่หน้าตู้ต่อจาก Primary Zone โดยวางเรียงเป็น คอลัมน์ตรงๆ ไม่มีการสลับ Pattern

เงื่อนไขการเข้าโซน Condo

- 1 สินค้านั้นยังมีกล่องค้างเหลือหลัง Primary
- 2 จำนวนที่เหลือ $\geq \text{cols} \times \text{targetLayers}$ (ต้องเต็ม 1 คอลัมน์พอดี)
- 3 มีพื้นที่ใน Y เพียงพอ ($\text{condoY} + \text{colDepth} \leq \text{InteriorL}$)



กล่องที่เหลือ ไม่พอ 1 คอลัมน์เต็ม จะไม่ถูกวางใน Condo แต่จะส่งต่อไป Scatter แทน

```
targetCondoZ = [ InteriorH / H_adjacent ] * H_adjacent
targetLayers = min( [ targetCondoZ / H_product ], [ InteriorH / H_product ] )
condoCap      = cols * targetLayers (กล่องต่อ 1 คอลัมน์)
```

ถ้ายังมีกล่องเหลือหลัง Primary + Condo จะนำไปวางทับยอดกองที่มีอยู่แล้ว เฉพาะกล่องสินค้าเดิมทับ
กองสินค้านั้น (ห้ามข้ามสินค้า)

อัลกอริทึมเลือกกอง

สร้างรายการ "กองที่รับได้" = กองที่ยังไม่เต็มเพดาน InteriorH

วนซ้ำจนกล่องหมดหรือทุกกองเต็ม:

- เลือกกองที่มี TopZ ต่ำสุด (สั้นที่สุด)
 - + TopZ เท่ากัน → เลือก StackIndex น้อยกว่า
- วางชั้นใหม่ที่ยอดกองนั้น (Pattern ตาม A/B ของกองนั้น)
- อัปเดต TopZ ของกองนั้น
- ถ้ากองเต็ม → นำออกจากรายการ

ตัวอย่าง 3 รอบ

รอบที่ 1:



[A] [B]

B สั้นกว่า

รอบที่ 2:



[A] [B]

A สั้นกว่า

รอบที่ 3:



[A] [B]

เท่ากัน → A (SI เล็กกว่า)



Scatter ไม่จำกัดด้วย MaxLayers — วางได้จนถึงเพดานตู้ (InteriorH) MaxLayers ส่งผลเฉพาะ
ในชั้น Primary เท่านั้น

05 รูปแบบการวางชั้น (Layer Pattern)

Layer Pattern Structure

แต่ละชั้นถูกกำหนดด้วย Pattern ซึ่งประกอบด้วย Section หลายส่วนวางต่อกันในแนวกว้าง (X) แต่ละ Section มี SubRow ที่กำหนดแถว คอลัมน์ และทิศทางการวาง

โครงสร้าง JSON

```
{
  "PatternA": [
    {
      "SubRows": [
        { "Rows": 2, "Cols": 6, "Rotated": false }, // 2 แถว × 6 คอลัมน์ ไม่หมุน
      ]
    },
    {
      "SubRows": [
        { "Rows": 3, "Cols": 3, "Rotated": true } // 3 แถว × 3 คอลัมน์ หมุน 90°
      ]
    }
  ]
}
```

Field	ความหมาย
Rows	จำนวนแถวในแนว Y (ความลึก)
Cols	จำนวนคอลัมน์ในแนว X (ความกว้าง)
Rotated = false	วางปกติ — ด้าน W ในแนว X, ด้าน L ในแนว Y
Rotated = true	หมุน 90° — ด้าน L ในแนว X, ด้าน W ในแนว Y

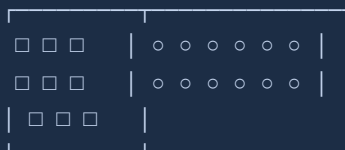
ตัวอย่าง: Aloe 365 ML (21.9 × 33.4 × 20.5 cm)

PatternA: [Section(2×6 ไม่หมุน), Section(3×3 หมุน)]
PatternB: [Section(3×3 หมุน), Section(2×6 ไม่หมุน)]

ชั้น 0 (PatternA):



ชั้น 1 (PatternB):



○ = ไม่หมุน □ = หมุน 90°

A/B สลับซ้าย-ขวา → แต่ละชั้นล็อกกันมั่นคง

ความจุต่อชั้น (Layer Capacity)

ระบบนับความจุโดย simulate การวางลงบน scratch list แล้วนับ:

```
capacity = PlaceLayerAt(sections, spec, dims, stackY, z=0,  
                        limit=∞, productIndex=-1, scratch, ...)
```

→ จำนวนกล่องที่วางได้จริงใน 1 ชั้น ณ ตำแหน่ง Y นั้น



ถ้ากล่องบางตัวล้นขอบตู้ (`px+bw > InteriorW` หรือ `py+bl > InteriorL`) จะถูกข้ามไป — ไม่ถูกนับและไม่ถูกวาง

Mogu 1000 ML × 540 + Mogu 320 ML × 850 ในตู้สั้น 20 ft

สินค้า	ขนาด W×L×H (cm)	MaxLayers	ต้องการ
Mogu 1000 ML	26.2 × 34.8 × 26.7	8	540 กล่อง
Mogu 320 ML	25.8 × 38.5 × 15.7	13	850 กล่อง

ชั้น 0 — ComputeEffectiveLayers

Mogu 1000: $bpl \approx 26$, $stacks = \lceil 540 / (26 \times 8) \rceil = 3$, $depth = 3 \times 34.8 = 104.4$ cm

Mogu 320: $bpl \approx 26$, $stacks = \lceil 850 / (26 \times 13) \rceil = 3$, $depth = 3 \times 38.5 = 115.5$ cm

$requiredY \approx 220$ cm < $targetY = 595 - 50 = 545$ cm  ไม่ต้องลด EffectiveLayers

ชั้น 1 — Primary Packing

Mogu 1000: วาง 3 กอง × 8 ชั้น → บรรจุครบ 540

Mogu 320: วาง 3 กอง × 13 ชั้น → บรรจุครบ 850

ชั้น 2-4 — Balancing

ปรับให้กองแต่ละกองของสินค้าเดียวกันมีจำนวนชั้นใกล้เคียงกัน ± 1 ชั้น

ชั้น 5 — Sort

กองที่มีชั้นน้อยกว่า → เลื่อนไปใกล้ประตูมากขึ้น

ชั้น 6-7 — Condo + Scatter

ถ้ายังมีกล่องเหลือหลัง Primary:

Condo → วางคอลัมน์ที่ $condoAreaStart$ จนหมดหรือไม่พอ 1 คอลัมน์เต็ม

Scatter → เอากล่องที่เหลือทั่วยอดกองเดิม เลือกกองสั้นสุดก่อน



ผลลัพธ์ที่คาดหวัง: Mogu 1000 ML ครบ 540, Mogu 320 ML ครบ 850 ทุกกล่องอยู่ภายในขอบตู้ ไม่มีกล่องสั้น ไม่มีการข้ามสินค้า

การรับประกันของ Algorithm



ไม่มีกล่องล้นตู้

ทุก BoxPlacement ถูกตรวจสอบให้อยู่ภายใน $\text{InteriorW} \times \text{InteriorL} \times \text{InteriorH}$ เสมอ



ไม่วางเกินจำนวนที่ต้องการ

$\text{Packed} \leq \text{Requested}$ ทุกสินค้า ระบบหยุดเมื่อถึงจำนวนพอดี



ชั้นต้องต่อเนื่อง

ไม่มีช่องว่างระหว่างชั้น ทุกกล่องมีฐานรองรับ
ไม่มีกล่องลอยอากาศ



สินค้าไม่ข้ามกอง

Scatter วางเฉพาะกล่องสินค้าเดิมทับกองของสินค้านั้นเท่านั้น

ลำดับความสำคัญ

ลำดับ	เป้าหมาย	วิธีการ
1st	วางกล่องให้ได้มากที่สุด	Primary → Condo → Scatter
2nd	ใช้พื้นที่ตัวอย่างสม่ำเสมอ	EffectiveLayers + Balancing
3rd	กองสั้นอยู่ใกล้ประตู (ขนง่าย)	Sort Stacks by Height
4th	ความสูงกองสินค้าเดียวกัน ใกล้เคียงกัน	Layer Balancing

ข้อจำกัดที่ทราบ



สินค้าที่ไม่มี Pattern จะไม่ถูกวางเลยในทุกชั้นตอน ต้องกำหนด PatternA ก่อน



Condo รองรับเฉพาะกล่องที่มีพอดเต็ม 1 คอลัมน์ ($\text{cols} \times \text{targetLayers}$) ถ้าน้อยกว่าจะใช้ Scatter แทน



Algorithm ไม่รองรับหลายสินค้าปนกันในกองเดียว — แต่ละกองมีสินค้าชนิดเดียว ตามความต้องการด้านการจัดการโกดัง

